Advanced C Curriculum Exercise

Source-code comprehension for purpose, architecture, and API-oriented use cases

Student: _____
Date: _____

Source under study: `curriculum_advanced.pdf`, which contains `configurator_v2.c`: a configurable C program that can generate outputs through command-line flags, config files, or a micro UI. The source supports flow types including `cartesian`, `literal`, `repeat`, and `reverse`, and abstracts output through a sink-like interface.

Estimated time: 60-90 minutes.
Prerequisites: C structs, enums, function pointers, file I/O, CLI parsing, bounded string handling, and integer overflow guards.

Learning objectives

- Explain the high-level purpose of `configurator_v2.c` without being distracted by `xN`-style internal identifiers.
- Trace how user-facing API inputs - CLI flags, config-file keys, and UI prompts - become an executable object specification.
- Identify core API-oriented design choices: stable command surface, config schema, output abstraction, validation boundary, and dispatch by flow type.
- Evaluate plausible use cases for generated outputs in API-oriented software engineering.
- Propose tests and design improvements that preserve the program's external contract while improving safety and maintainability.

Read this orientation before answering

The program's internal names are intentionally opaque (`x3`, `x28`, `x167`, etc.), but the external interface is comparatively clear. You should read from the outside inward: usage text, supported flags, config format, parsing functions, validation, flow dispatch, and output writing. Treat the user-facing flags and config keys as the program's API surface.

Concept	Where to look	What to infer
Object specification	`x28`, default init, key application	The runtime object carries names, input payload, width, flow, target, separator, prefix, and suffix.
Fabric	`x31`, add object, execute collection	A config file can hold multiple object specifications executed in sequence.
Output sink	`x3`, file/stdout init, write helpers	The generator is decoupled from the destination; file and stdout share the same write contract.
Flow dispatch	Flow enum and execution branch	The same object schema can run cartesian, literal, repeat, or reverse behavior.

Concept	Where to look	What to infer
Validation boundary	numeric parser, safe power, object validation	Bad widths, empty names, overflows, and impossible combinations should fail before generation.

Part A - Purpose comprehension

A1. In one sentence, state the program's purpose from the viewpoint of a user running it from a shell.

> Answer:

A2. In one sentence, state the program's purpose from the viewpoint of an API or platform engineer.

> Answer:

A3. What problem does the program solve better than a hard-coded permutation generator? Name two differences.

> Answer:

A4. Why might the author preserve opaque `xN` internal identifiers while making runtime object names user-specified?

> Answer:

Part B - External API surface

Fill in the table. Treat every flag or config key as part of the program's API contract.

API input	Accepted examples or aliases	Effect on execution	Validation concern
`object_name`			
`input_value`			
`input_width`			
`flow_type`			
`output_target`			
`separator` / `prefix` / `suffix`			

Part C - Architecture trace

Draw arrows or write a numbered path showing how a value moves from user input to output. Use the function and structure names even if they are opaque.

Example path to complete:

```
CLI flag or config line -> parser -> object specification -> validation -> flow
function -> output sink -> file/stdout
```

C1. Trace:

```
--input-value 01 --input-width 3 --flow-type cartesian --output-target out.txt
```

> Answer:

C2. Trace a config-file object that uses `flow_type=reverse` and `output_target=stdout`.

> Answer:

C3. Explain where the program switches from declarative configuration to imperative execution.

> Answer:

Part D - Code-reading focus questions

D1. Bounded copy: What does the helper that copies strings into fixed buffers prevent, and what usability trade-off does it create?

> Answer:

D2. Numeric parsing: Why does the width parser reject negative values and trailing junk?

> Answer:

D3. Escape decoding: What user-facing feature is enabled by translating `\n`, `\t`, `\r`, and `\\`?

> Answer:

D4. Overflow guard: Why must cartesian generation calculate `strlen(input_value) ^ input_width` safely before iterating?

> Answer:

D5. Output abstraction: How does the sink interface make the generator more testable or extensible?

> Answer:

D6. Multi-object config: What is the engineering significance of supporting more than one object in a config file?

> Answer:

Part E - Multiple choice diagnostic

Choose one answer for each. Do not use the answer key until you have written your choices.

E1. Which statement best describes the external API of this program?

- A. The `xN` function names, because they are the only stable user interface.
- B. The CLI flags, config keys, UI prompts, accepted flow names, and output targets.
- C. The local variable names inside each function.
- D. The compiler optimization level.

Choice: ___
Reason: _____

E2. Why is the output sink design API-oriented?

- A. It hides every error from callers.
- B. It couples generation directly to `fopen`.
- C. It defines a small write contract that different destinations can implement.
- D. It prevents stdout output.

Choice: ___
Reason: _____

E3. What is the main purpose of validating a cartesian object's width and input before generation?

- A. To make output alphabetical.
- B. To prevent impossible, overflowing, or meaningless generation work.
- C. To force every flow to write a file.
- D. To remove the need for tests.

Choice: ___
Reason: _____

E4. Which use case best fits the program as written?

- A. Generating test payloads or fixture files from a declared input alphabet and flow.
- B. Replacing an HTTP server framework.
- C. Encrypting API keys at rest.
- D. Proving memory safety automatically.

Choice: ___
Reason: _____

E5. What does support for config files add beyond one-off CLI flags?

- A. A repeatable declarative artifact that can be versioned, reviewed, and run again.
- B. Guaranteed faster generation for every input.
- C. Elimination of all validation logic.
- D. Removal of stdout support.

Choice: ___
Reason: _____

Part F - API-oriented use-case design

Design three realistic use cases. Each use case must name: user persona, API surface used, flow type, output target, and risk/limit.

Use case	Persona	Interface	Flow	Output	Risk/limit
1					
2					
3					

Part G - Design review and improvement

G1. Identify two strengths of the implementation for API-oriented engineering.

> Answer:

G2. Identify two weaknesses or maintainability risks. At least one must relate to `xN` identifiers, fixed buffer sizes, or error reporting.

> Answer:

G3. Propose one backward-compatible change that improves the API contract. Explain why it does not break existing users.

> Answer:

G4. Propose one non-backward-compatible change that might still be worthwhile in a v3 release.

> Answer:

Part H - Test plan

Write a small test plan. Include at least one unit-level test, one integration test, one failure-path test, and one boundary test.

Test type	Input	Expected result	Why it matters
Unit			
Integration			
Failure path			
Boundary / overflow			
Regression			

Part I - Constructed response

I1. Write a 250-350 word explanation answering this prompt:

> Explain the purpose of `configurator_v2.c` and evaluate how it could be used in API-oriented software engineering.

Your answer must discuss the CLI/config/UI surfaces, the object specification, flow dispatch, output abstraction, validation, and at least two concrete use cases.

Scoring rubric for Part I - 20 points

Criterion	Points	Evidence of mastery
Purpose	4	Accurately describes a user-specifiable generator/fabric rather than a single hard-coded algorithm.
API surface	4	Explains CLI flags, config keys, micro UI, accepted flow names, and outputs as external contract.
Architecture	4	Connects object spec, validation, flow dispatch, and output sink.
Use cases	4	Gives plausible API-engineering uses such as fixture generation, payload combinatorics, config-driven batch generation, docs/demo artifacts, or golden-file testing.
Critical evaluation	4	Mentions risks or improvements: naming opacity, bounded buffers, overflow limits, clearer diagnostics, extensible sink registration, or machine-readable errors.

Answer key and instructor notes

Use this section after completing the worksheet.

Part A - expected themes

- User view: a command-line/configurable tool that writes generated text outputs based on a named object, input payload/alphabet, width, flow type, and output target.
- API/platform view: a small declarative generation engine whose external contract is flags, config keys, supported flows, and output destinations.
- Better than a hard-coded generator: it supports multiple flows, config files, stdout/files, prefixes/suffixes/separators, validation, and multi-object batches.
- Opaque internal names can preserve compatibility with an earlier naming convention while allowing meaningful runtime names for users, though this harms maintainability.

Part B - sample completions

Input	Accepted examples / aliases	Effect	Validation concern
`object_name`	`--object-name`, `-n`, `object`, `name`	Names the runtime object in the spec.	Must fit fixed buffer and not be empty.
`input_value`	`--input-value`, `-i`, `input`, `alphabet`	Payload or alphabet used by the selected flow.	Must fit buffer; cartesian cannot use empty input.
`input_width`	`--input-width`, `-w`, `width`, `length`, `depth`	Depth/count/width for generation.	Reject negatives, junk, zero; cartesian bounded by buffer and overflow guard.
`flow_type`	`cartesian`/`product`/`combinations`, `literal`/`echo`, `repeat`, `reverse`	Selects generation behavior.	Unknown values fail.
`output_target`	`--output-target`, `-o`, `output`, `file`, `stdout`, `-`	Chooses file path or stdout sink.	File open can fail; target must not be empty.
`separator`/`prefix`/`suffix`	`--separator`, `--prefix`, `--suffix`, escapes	Controls record formatting around each generated item.	Escaped text must fit destination buffers.

Part C/D - architecture notes

- CLI/config/UI all populate an `x28` object specification. A collection of objects is stored in `x31`.
- Validation happens before execution; cartesian generation checks buffer-relevant width, non-empty input, and safe power calculation.
- The flow parser maps user strings to enum values; execution dispatch calls cartesian, literal, repeat, or reverse generation.
- The write path goes through a sink object with function pointers for string and character writes, allowing file and stdout behavior through one contract.

- Config-file support makes the program closer to an API automation primitive: configurations can be checked into version control, reviewed, rerun, and used by CI jobs.

Part E - MCQ answers with rationales

E1: B. The external API is the user-facing contract: flags, config keys, UI prompts, accepted names, and targets. The `xN` internals are implementation details.

E2: C. A sink is a small interface that can be implemented by files, stdout, memory buffers, network adapters, or test doubles.

E3: B. Cartesian output can explode combinatorially; validation prevents impossible, overflowing, or meaningless work.

E4: A. The existing behavior is well suited to payload, fixture, and golden-file generation. It is not a server framework, encryption system, or formal verifier.

E5: A. A config file is a repeatable declarative artifact. It improves reviewability, versioning, and automation compared with ad hoc shell commands.

Part F - plausible use cases

- API test fixture generation: use cartesian flow to produce combinations of allowed path/query/header characters for endpoint tests.
- Golden-file generation: use literal, repeat, or reverse flows to create deterministic outputs checked into tests.
- Documentation/demo generation: create sample payload files from config so examples stay repeatable.
- Contract fuzzing seed generation: produce bounded token combinations for validating parsers, provided output size is controlled.
- Batch artifact generation: define multiple objects in one config and run them as a small generation pipeline.

Part G/H - review and testing notes

- Strengths: declarative config, stable CLI surface, sink abstraction, explicit validation, bounded copies, overflow checks, multiple flow modes.
- Weaknesses: opaque internal names, fixed buffer truncation/failure behavior, limited diagnostics, no plugin registry for flows/sinks, no structured machine-readable errors.
- Backward-compatible improvement: add `--dry-run`, `--json-errors`, or a memory sink for tests without removing existing flags.
- Possible v3 breaking change: replace `xN` internal identifiers with descriptive names, reorganize objects into public/private headers, or define a formal config schema.
- Useful tests: parser accepts aliases; parser rejects negative width; cartesian 2 symbols width 3 creates 8 records; stdout target does not close stdout; overflow guard rejects unsafe powers; config with two objects executes both.

Source note

Exercise based on the uploaded advanced curriculum file `curriculum_advanced.pdf` and the Learning Engine instruction-set expectations for comprehension questions, MCQ rationales, constructed response, rubric, and mastery-oriented review.